Study Notes: Claude Code Architecture, Skills & Multi-Agent Orchestration

1. TL;DR / Executive Summary

- **Modular Configuration:** Claude Code uses .clauderc directories to manage instructions, requiring a single source of truth to prevent context duplication and token bloat.
- **Progressive Disclosure in Skills:** Skills package procedural knowledge into skill.md files, loading into context on-demand only when triggered via front-matter descriptions.
- **Multi-Agent Orchestration:** The main Claude instance acts as an orchestrator delegating sub-tasks to specialized agents, preserving context, enabling parallel efficiency, and enforcing safety boundaries.
- **Security & Best Practices:** Emphasizes rigorous code auditing of third-party skills (e.g., from skss.io), iterative testing, and standardization.

2. Core Concepts & Definitions

Term / Concept	Definition / Explanation
.claude Directory	The project-level configuration structure managing instructions, agent definitions, and skills.
Agent Skills	Reusable packages containing procedural knowledge (skill.md and optional references/) loaded progressively on-demand.
Progressive Disclosure	A performance optimization where Claude loads only the necessary skill body when triggered rather than bloating the main context window.
Multi-Agent Orchestration	A system architecture where a primary Claude instance delegates specialized tasks (research, coding, debugging) to sub-agents.
Skill Registry	A platform repository where developers can discover and download community-created skills.
Skill Creator	A standardization tool used to generate consistent skills

	compatible with Claude's architecture.

3. Structured Body Hierarchy

- **Project Configuration & Context Management**
  - **Single Source of Truth:** Avoid duplicating instructions across files like clauderc.md and agents.md to prevent token bloat and conflicting agent behavior.
  - **Modular References:** Utilize reference links (e.g., @agent.md) to maintain clean file separation and keep Claude context-aware and efficient.
- **Understanding and Implementing Skills**
  - **Anatomy & Structure:** Consists of a skill.md file featuring YAML front matter (name, description, version) and an optional references/ directory.
  - **Triggering & Loading:** Claude triggers skills using front-matter descriptions, leveraging progressive disclosure to keep context windows clean.
  - **Third-Party Security:** Because skills can execute local host code, developers must audit external registry downloads (e.g., skss.io) before execution.
- **Multi-Agent Systems and Orchestration**
  - **Orchestration Model:** Main Claude instance manages and delegates tasks to specialized sub-agents.
  - **Key Benefits:**
    - *Context Preservation*: Keeps sub-task details out of the main conversation window.
    - *Efficiency:* Allows parallel execution of sub-agents for faster delivery.
    - *Safety*: Establishes strict behavioral boundaries and limitations for sub-agents.
- **Best Practices for Development**
  - **Audit & Verify:** Check vendor credibility, download counts, and security warnings before running external code.
  - **Iterative Testing:** Test skills against specific use cases and refine instructions using feedback loops.
  - **Standardization & Tooling:** Use the Skill Creator tool for reliable formatting; leverage CLI or Desktop apps interchangeably since they share the same underlying logic.

4. Review & Active Recall (Q&A)

- **Q: Why is avoiding instruction duplication across files like clauderc.md and agents.md critical?**
  - **A:** Duplication causes token bloat, wastes context window space, and risks introducing conflicting instructions that confuse the agent.
- **Q: How does progressive disclosure optimize performance in Claude Code?**
  - **A:** Instead of loading all procedural data upfront, Claude keeps the context clean by fetching and loading the full body of a skill only when it is explicitly or implicitly triggered.
- **Q: What are the primary architectural benefits of multi-agent orchestration?**
  - **A:** It preserves main conversation context, enables parallel task execution for speed, and enforces safety boundaries on specialized sub-agents.
- **Q: What safety precautions should be taken when acquiring skills from public registries like skss.io?**
  - **A:** Developers must perform a local code audit, verify vendor credibility, inspect installation metrics, and check for security warnings since these agents can execute scripts on the host machine.